

Savings-heuristic algorithm for Travelling-salesman problem (Citation-references 1., 2. & 3.)

(This heuristic constructive-search algorithm is taught in 'Week-3 Lecture-4 minute-16' of the A.I.-S.M.P.S subject in this I.I.T.-M B.Sc.-data-science course.) ¹.

(It is also explained in 'Deepak Khemani. A First Course in Artificial Intelligence, McGraw Hill Education (India), 2013, Chapter-4.4.1': Constructive-methods for the Travelling-salesman problem.) ².

(It was first published in the paper : Clarke, G., & Wright, J. W. (1964). Scheduling of vehicles from a central depot to a number of delivery points. Operations Research, 12(4), 568-581.) ³.

Main steps in the savings-heuristic algorithm for Travelling-salesman problem :

1. First, all the nodes are sorted in ascending order of total edge-costs from each node. (sum of all column-values in a row of the cost-matrix.)
2. Then, the savings-heuristic algorithm is run iteratively, using different nodes as the starting-node.
3. The lowest tour-cost valid-tour generated from all the iterations, is found.

The initial iterative executions of the algorithm use the sorted nodes-list for starting-node. But then in subsequent iterations, it sometimes uses randomly-selected starting-nodes to introduce an element of **stochastic/random-exploration** ⁴. in the search-algorithm strategy.

(The concept of stochastic/random exploration in search-strategy, is taught in **Week-4 Lecture-2 minute-2** of the A.I.-S.M.P.S subject in this I.I.T.-M B.Sc.-data-science course.) ⁴.

In certain iterations, the starting-node is also selected from the middle or bottom of the sorted nodes-list.

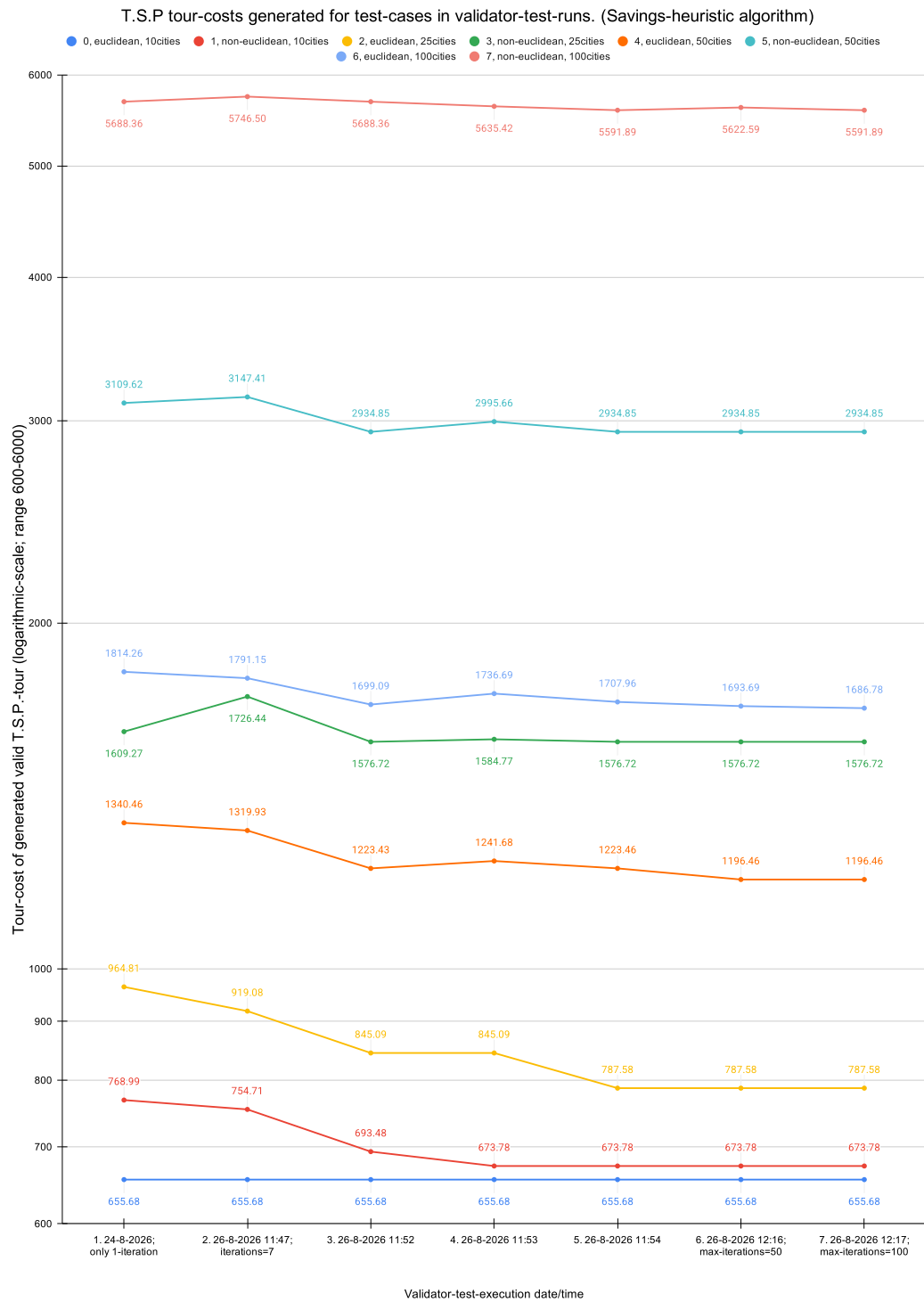
The same savings-heuristic algorithm works for both euclidean and non-euclidean travelling-salesman problems.

A **maximum-iterations-limit (150)** and **maximum-execution-time-limit (50 seconds)** are also set in the script, for handling huge input data-sets and test-cases. So that the program stops execution with atleast one valid-tour solution, and doesn't keep running for too long.

Test-run results for the 8 test-cases given in the validator for travelling-salesman problem.

Test-case runs	0, euc_10	1, non-euc_10	2, euc_25	3, non-euc_25	4, euc_50	5, non-euc_50	6, euc_100	7, non-euc_100
1. 24-8-2026; only 1-iteration	655.6751331590185	768.988361117	964.812249689	1609.268829199	1340.460459118	3109.62367178	1814.262870419	5688.3586401872
2. 26-8-2026 11:47; iterations=7	655.6751331590184, (7)	754.71076386199, (1)	919.082356725, (17)	1726.442756836, (17)	1319.92637077, (17)	3147.410918719, (6)	1791.15178762, (85)	5746.500427348, (69)
3. 26-8-2026 11:52	655.6751331590184, (5)	693.48129862649, (2)	845.0878576714, (11)	1576.720440208, (9)	1223.4319945288, (48)	2934.84797799, (8)	1699.094389369, (88)	5688.3586401872, (20)
4. 26-8-2026 11:53	655.6751331590184, (5)	673.7787665244, (9)	845.0878576714, (11)	1584.7737918, (4)	1241.6814249, (20)	2995.65760728, (33)	1736.687648228, (34)	5635.42056207, (0)
5. 26-8-2026 11:54	655.6751331590184, (5)	673.7787665244, (9)	787.5752663189, (24)	1576.720440208, (9)	1223.4572162, (29)	2934.84797799, (8)	1707.964491164, (59)	5591.8855703, (3)
6. 26-8-2026 12:16; max-iterations=50	655.6751331590184, (5)	673.7787665244, (9)	787.5752663189, (24)	1576.720440208, (9)	1196.460993216, (26)	2934.84797799, (8)	1693.69099236, (22)	5622.59455523, (81)
7. 26-8-2026 12:17; max-iterations=100	655.6751331590184, (5)	673.7787665244, (9)	787.5752663189, (24)	1576.720440208, (9)	1196.460993216, (26)	2934.84797799, (8)	1686.784307951, (99)	5591.8855703, (3)

The starting-node selected for each tour-generation in the above results, is mentioned inside paranthesis () next to each tour-cost.



It is observed in the test-runs listed above, that the lowest-cost valid-tours are found after greater number of iterations on different starting-nodes.

The best and lowest-cost valid-tours for all test-cases are found in the final test-run (**No. 7**) listed above, where the savings-heuristic algorithm was iteratively run with all given nodes as the starting-node in each test-case. (The maximum-iteration limit was set to 100 in the final test-run listed above.)

These results indicate that the node with the lowest total-edge-cost to all other nodes in a T.S.problem, (node closest to the geometric-center of all the node-locations in euclidean T.S.P.), is not necessarily the best starting-node for generating the lowest-cost valid-tour using the savings-heuristic algorithm.

Savings-heuristic algorithm logic :

1. After the starting-node is selected, all the edges between all the other nodes are sorted in decreasing-order of **edge-cost savings** compared to edges with the starting-node.

(cost-savings of edge i-to-j) = (cost of i-to-startnode) + (cost of startnode-to-j) - (cost of i-to-j) ; (Reference 1.)

2. Then the valid-tour is generated by adding the highest cost-saving edges one by one, without creating sub-loops or closing the tour prematurely.

To check for and prevent sub-loops in the generated valid-tour, two new columns are appended to the cost-matrix. (in-degree and out-degree of edges to each node should not exceed two)

3. The total cost of the generated valid-tour is compared to find the minimum-cost valid-tour for each test-case.

Time/Space-complexities of this algorithm-implementation :

1. The space-complexity of this savings-heuristic algorithm implementation is : **$O(n^2)$** .

The 'n x n' cost-matrix takes $O(n^2)$ space; The edge-cost-savings list has '2 x n^2 ' values (both directions for each edge), and takes $O(n^2)$ space.

2. The time-complexity of this savings-heuristic algorithm implementation is : **$O(n^2 \log_2 n)$** .

Creating the 'n x n' cost-matrix takes $O(n^2)$ time; The edge-cost-savings list takes $O(n^2)$ time; The cost-savings list of '2 n^2 ' values is sorted using Timsort (divide-&-conquer strategy) and this takes $O(n^2 \log_2 n)$ time; Constructing a valid-tour of n-nodes after checking for any sub-loops, takes $O(n^2)$ time.

Because the maximum-iterations-limit is set to 150, repeated executions of the entire algorithm for a T.S.Problem doesn't affect the $O()$ space/time-complexity of the program.

My intuitive-idea for future exploratory-analysis in T.S.P.

1. Create a new version of the savings-heuristic algorithm for T.S.P, in which the valid-tour generation in the algorithm uses the second-highest cost-saving edges instead of the highest cost-saving edges, for each new edge of the valid-tour.
2. Occasionally, a highest cost-saving edge can also be selected as a new edge in the valid-tour, during this iterative edge-addition process for valid-tour generation.
3. Run this new version on the given test-cases input data-sets, and compare tour-cost results obtained with previous results.

References

1. Week-3 Lecture-4:'Solution-space search', minute-16, Course-ID: BSCS3003, '**A.I.- Search-methods in problem-solving**', Prof. Deepak Khemani
2. Deepak Khemani. '**A First Course in Artificial Intelligence**', McGraw Hill Education (India), 2013; Chapter-4.4.1: 'Constructive-methods for the Travelling-salesman problem', p94-p95.
3. Clarke, G., & Wright, J. W. (1964). '**Scheduling of vehicles from a central depot to a number of delivery points**'. Operations Research, 12(4), 568-581.

4. Week-4 Lecture-2:'Stochastic local search', minute-2, Course-ID: BSCS3003, 'A.I.- Search-methods in problem-solving', Prof. Deepak Khemani

5. Validator test-results line-chart visualisation in google-sheets; https://docs.google.com/spreadsheets/d/1yp_jYd30vz6CLIG34jS72RvESLR81yqn4IaCfC2dEnM/

Declaration :

I have done this programming-assignment on my own, to implement a solution for the Travelling-salesman problem in the Python-language, using my learning and understanding of the savings-heuristic algorithm taught in this 'A.I.- S.M.P.S.' subject.

I have used proper reference-citations for all search-concepts and ideas learnt and applied by me to complete and submit this programming-assignment work. (Including the Savings-heuristic algorithm used and mentioned above.)

In initial trial-submission versions of this assignment report, I didn't include the citation-referencing for the savings-heuristic algorithm used in my assignment-work. I have included them now in my final-submission here.

Student-id: 21f3002389; George